

## 第 04 讲

### 实验设计：从手写矩阵乘法到 Transformer Q/K/V 投影

实验目标：把“行 dot 列”“Shape 兼容规则”“Batch 线性映射”和  $Q = XW_Q$  变成可以观察、验证和解释的实验现象。

实验设计文档

## 1 实验定位

本实验不是为了训练一个大模型，而是建立**可验证的底层数学直觉**。实验采用科研式闭环：

问题 → 预测 → 操作 → 结果 → 解释 → 结论

### 科研视角

从第一阶段开始就培养一个习惯：实验前先写预测，实验后再解释差异。未来做 Agent Systems 研究时，实验规模会大很多，但科学推理结构仍然相同。

## 2 实验环境

- Python 3.x;
- NumPy;
- PyTorch（可选，本环境实验已验证安装可用）;
- 不需要 GPU;
- 实验代码：lesson04\_matrix\_multiplication\_experiment.py。

运行：

```
python lesson04_matrix_multiplication_experiment.py
```

## 3 实验总问题

1. 矩阵乘法的一个输出元素是否真的等于“行 dot 列”？
2. 为什么  $(m, n) @ (n, p)$  才能相乘？
3. 矩阵乘法和逐元素乘法到底有什么数值差异？
4. 为什么神经网络可以用  $XW$  一次处理整个 Batch？
5. Transformer 中  $Q = XW_Q$ 、 $K = XW_K$ 、 $V = XW_V$  是否只是同一矩阵规则的不同权重版本？

## 4 实验 1：手算矩阵乘法并用 NumPy 验证

### 研究问题

对于：

$$A : (2, 3), \quad B : (3, 2)$$

NumPy 的  $A @ B$  是否与我们按“行 dot 列”逐格手算完全一致？

## 假设

若矩阵乘法定义正确，则：

$$C_{ij} = A_{i,:} \cdot B_{:,j}$$

手算、手写循环和 NumPy 应得到完全相同的  $2 \times 2$  结果。

## 实验数据

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$

## 操作步骤

1. 先在纸上预测输出 Shape；
2. 手算四个  $C_{ij}$ ；
3. 运行 `manual_matmul(A,B)`；
4. 运行 `A @ B`；
5. 使用 `np.allclose` 比较结果。

## 预期结果

$$C = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}$$

两种程序计算与手算一致。

## 5 实验 2：只根据 Shape 预测输出

### 研究问题

不看具体数值，仅根据：

$$X : (4, 3), \quad W : (3, 5)$$

能否预测  $XW$  的 Shape？

## 假设

按照：

$$(m, n) @ (n, p) \rightarrow (m, p)$$

应该得到：

$$(4, 5)$$

## 观测变量

- 自变量： $X$  和  $W$  的 Shape；
- 因变量：输出  $Y.shape$ ；
- 重点不是数值大小，而是 Shape 规则是否成立。

## 6 实验 3：故意制造 Shape 不匹配

### 研究问题

为什么  $A : (2, 3)$  不能直接与  $B : (2, 4)$  做矩阵乘法？

### 假设

由于 3 与 2 不匹配，行与列无法形成等长 Dot Product，因此 NumPy 应抛出矩阵乘法维度错误。

### 方法

代码使用 try/except 捕获错误，保证整个实验脚本不会因为这个“预期失败”而中断。

#### 常见误区 / 风险

这是一个非常重要的实验观念：**预期失败也可以有效实验结果**。如果我们故意构造违反条件的输入，那么报错本身验证了规则边界。

## 7 实验 4：矩阵乘法与逐元素乘法对照

使用两个  $2 \times 2$  矩阵，同时执行：

```
M1 * M2
M1 @ M2
```

## 控制思路

输入完全相同，只改变运算符。这样可以把结果差异归因于“运算定义不同”。

## 预期现象

\* 只做对应位置相乘；@ 做行与列的 Dot Product，所以数值完全不同。

## 8 实验 5：Batch 如何一次通过权重矩阵

定义：

$$X_{batch} : (3, 2)$$

表示 3 个样本，每个样本 2 个输入特征。

定义：

$$W : (2, 4)$$

表示从 2 个输入特征映射到 4 个输出特征。

预测：

$$(3, 2) @ (2, 4) \rightarrow (3, 4)$$

## 解释任务

运行后不要只看数字。请逐句回答：

1. 输出为什么仍有 3 行？
2. 为什么每行从 2 个数变成 4 个数？
3. 第一个样本的 4 个输出，分别对应什么 Dot Product？

## 9 实验 6：微型 Transformer 的 Q/K/V 投影

构造：

$$X : (4, 3)$$

代表 4 个 Token，每个 Token 只有 3 维 hidden state。

再定义三套不同参数：

$$W_Q, W_K, W_V : (3, 2)$$

计算：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

## 假设

三者 Shape 都应为：

$$(4, 2)$$

但由于权重矩阵不同，具体数值应不同。

## 关键解释

这一实验验证：Q/K/V 不是把 X “复制三份”，而是用三套不同可学习矩阵对同一个 X 做三种不同的线性投影。

## 10 实验 7：用 PyTorch 进行独立核对

将相同  $A$ 、 $B$  转成 `torch.Tensor`：

```
A_t = torch.tensor(A, dtype=torch.float32)
B_t = torch.tensor(B, dtype=torch.float32)
C_t = A_t @ B_t
```

若 PyTorch 与 NumPy 结果一致，说明我们理解的是通用数学规则，而不是某个 NumPy API 的特殊行为。

## 11 实际实验结果摘要

本材料包中的代码已经实际执行。关键结果：

- 手算结果：[[58, 64], [139, 154]]；
- 手写 `manual_matmul` 与 NumPy 完全一致；
- $(4, 3) @ (3, 5)$  实际输出 Shape 为  $(4, 5)$ ；
- 故意构造的不匹配 Shape 成功触发预期错误；
- Batch 线性映射得到  $(3, 4)$ ；
- 微型 Q/K/V 均由  $(4, 3) @ (3, 2)$  得到  $(4, 2)$ ；
- PyTorch 与 NumPy 矩阵乘法结果一致。

完整控制台输出见：第 04 讲 \_ 实验实际输出.txt。

## 12 实验记录表

建议你运行前先填写“预测”，运行后再填写“实际”。

| 实验         | 我的预测 | 实际结果 | 是否一致 / 原因 |
|------------|------|------|-----------|
| 实验 1 手算    |      |      |           |
| 实验 2 Shape |      |      |           |
| 实验 3 报错    |      |      |           |
| 实验 4 两种乘法  |      |      |           |
| 实验 5 Batch |      |      |           |
| 实验 6 Q/K/V |      |      |           |

### 13 科研式结论模板

完成实验后，可以用下面的结构自己写 5 句话：

1. 我验证的问题是：\_\_\_\_\_
2. 我原先预测：\_\_\_\_\_
3. 实际观察到：\_\_\_\_\_
4. 结果支持 / 不支持假设，因为：\_\_\_\_\_
5. 这对理解 Transformer 的意义是：\_\_\_\_\_

#### 本节检查点

最低验收：不运行代码，你也能预测每个实验中的输出 Shape；运行代码后，你能解释每一个 Shape，而不是只确认“程序没有报错”。